

diffblas: algorithmically differentiated BLAS routines

Sri Hari Krishna Narayanan ^{1¶}, Alexis Montoisson ¹, and Jean-Luc Bouchot ²

¹ Mathematics and Computer Science Division, Argonne National Laboratory, USA ² Inria de Saclay, Palaiseau, France ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

diffblas is a library that provides BLAS routines algorithmically differentiated using algorithmic differentiation principles (Griewank & Walther, 2008) from their reference implementations in LAPACK on GitHub using the automatic differentiation tool Tapenade (Hascoët & Pascual, 2013). It supports four modes: tangent (_d), vector tangent (_dv), adjoint (_b), and vector adjoint (_bv).

In addition to differentiating the standard Fortran-style BLAS interface, diffblas also provides differentiated CBLAS routines, facilitating interoperability with C and other languages. Its API mirrors BLAS / CBLAS, with additional arguments specifying differentiation variables, making integration into existing workflows straightforward.

diffblas relies on the underlying standard BLAS implementation and is agnostic to the backend (OpenBLAS, BLIS, MKL, Apple Accelerate, or libblastrampoline), ensuring both high performance and portability. Precompiled artifacts are available on GitHub, simplifying installation and use across different programming environments.

By providing efficient and accurate derivatives of linear algebra operations, diffblas facilitates gradient-based optimization, sensitivity analysis, and derivative-based workflows in scientific computing.

Statement of need

Linear algebra routines such as those in LAPACK are widely used in scientific computing, optimization, and machine learning. However, they do not provide derivatives, which are often required for gradient-based algorithms.

Existing approaches rely on hand-coded derivatives or generic automatic differentiation applied to high-level code, which can be inefficient or error-prone (Jonasson et al., 2020).

diffblas addresses this gap by providing algorithmically differentiated BLAS routines directly from reference LAPACK implementations and following relevant differentiation rules (Giles, 2008).

State of the field

Automatic source-to-source differentiation tools, such as Tapenade (Hascoët & Pascual, 2013), ADOL-C (Griewank et al., 1996), or TAF (Giering & Kaminski, 2003), provide general mechanisms to compute derivatives of code. However, naively applying AD to low-level BLAS or LAPACK routines can be inefficient due to loop structures, memory layout, and caching constraints (Jonasson et al., 2020). diffblas addresses these challenges by

38 generating differentiated routines directly from reference LAPACK implementations, combining
39 the reliability and performance of LAPACK with the efficiency of algorithmic differentiation.

40 Research impact statement

41 This work was inspired in part by a need to differentiate a Fortran code (Hascoët et al.,
42 2026) that uses BLAS and LAPACK routines, and to use the differentiated application for
43 gradient-based optimization.

44 Providing both the standard and CBLAS interfaces ensures that diffblas can be adopted across
45 different programming environments, facilitating derivative computations in diverse scientific
46 computing projects. Precompiled artifacts on GitHub further simplify integration, enabling
47 rapid deployment in multiple languages and scientific computing projects.

48 Acknowledgements

49 This work was supported in part by the Applied Mathematics activity within the U.S. Department
50 of Energy, Office of Science, Office of Advanced Scientific Computing Research Applied
51 Mathematics, and Office of Nuclear Physics SciDAC program under Contract No. DE-AC02-
52 06CH11357. This work was supported in part by NSF CSSI grant 2104068.

53 AI usage disclosure

54 Generative AI was used to ... ChatGPT was used to check spelling, grammar, and clarity of the
55 English text in this paper.

56 References

- 57 Giering, R., & Kaminski, T. (2003). Applying TAF to generate efficient derivative code of
58 Fortran 77-95 programs. *PAMM: Proceedings in Applied Mathematics and Mechanics*, 2,
59 54–57. <https://doi.org/10.1002/pamm.200310014>
- 60 Giles, M. (2008). *An extended collection of matrix derivative results for forward and reverse*
61 *mode automatic differentiation*.
- 62 Griewank, A., Juedes, D., & Utke, J. (1996). Algorithm 755: ADOL-C: a package for
63 the automatic differentiation of algorithms written in C/C++. *ACM Transactions on*
64 *Mathematical Software (TOMS)*, 22(2), 131–167. <https://doi.org/10.1145/229473.229474>
- 65 Griewank, A., & Walther, A. (2008). *Evaluating derivatives: Principles and techniques of*
66 *algorithmic differentiation*. SIAM. <https://doi.org/10.1137/1.9780898717761>
- 67 Hascoët, L., Menickelly, M., Narayanan, S. H. K., O’Neal, J., Schunck, N., & Wild, S. M.
68 (2026). HFBTHO-AD: Differentiation of a nuclear energy density functional code. *Computer*
69 *Physics Communications*, 320, 109955. <https://doi.org/10.1016/j.cpc.2025.109955>
- 70 Hascoët, L., & Pascual, V. (2013). The Tapenade automatic differentiation tool: principles,
71 model, and specification. *ACM Transactions on Mathematical Software (TOMS)*, 39(3),
72 1–43. <https://doi.org/10.1145/2450153.2450158>
- 73 Jonasson, K., Sigurdsson, S., Yngvason, H. F., Ragnarsson, P. O., & Melsted, P. (2020).
74 Algorithm 1005: Fortran subroutines for reverse mode algorithmic differentiation of blas
75 matrix operations. *ACM Transactions on Mathematical Software (TOMS)*, 46(1), 1–20.
76 <https://doi.org/10.1145/3382191>